

Redes Neuronales

U4 Tópicos Avanzados

Héctor Maravillo

Section 1

Kolmogorov's Theorem

Introduction

One use of a **neural network** is to **approximate a continuous mapping** f .

Since there are very simple mappings that a **single-layer net** cannot represent it is natural to ask how well a **multilayer net** can do.

The answer is given by the **Kolmogorov mapping neural network existence theorem** (1957), which states that a **feedforward** neural network with **three layers** of neurons (input units, hidden units, and output units) can represent **any continuous function** exactly.

Kolmogorov's theorem

The theorem its origins at the end of the nineteenth century when the mathematician Hilbert compiled a list of 23 unsolved problems as a challenge for twentieth century mathematicians (**Hilbert's problems**).



(a) David Hilbert



(b) Andréi
Kolmogórov

Kolmogorov–Arnold representation theorem

Hilbert's **thirteenth problem** concerns the issue of whether **functions of several variables** can be represented in terms of **superpositions of functions of fewer variables**.

He **conjectured** that there exist continuous functions of three variables which **cannot be represented** as superpositions of functions of two variables.

The conjecture was disproved by Vladimir Arnold in 1957.

Kolmogorov–Arnold representation theorem

However, a much more general result was obtained by Kolmogorov who showed that **every continuous function of several variables** (for a **closed and bounded input domain**) can be **represented** as the **superposition** of a small number of functions of one variable.

$$f(\mathbf{x}) = f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

where $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ and $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$.

Improved versions of Kolmogorov's theorem have been given by Sprecher (1965), Kahane (1975) and Lorentz (1976).

Implications for neural networks

In neural network terms this theorem says that any continuous mapping $y(x)$ from d input variables x_i to an output variable y can be **represented exactly** by a **three-layer neural network** having $d(2d + 1)$ units in the first hidden layer and $(2d + 1)$ units in the second hidden layer.

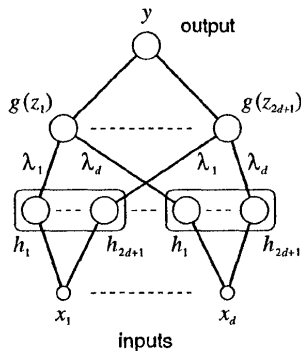


Figure: Network topology to implement Kolmogorov's theorem

Implications for neural networks

Each unit in the first hidden layer computes a function of one of the input variables x_i given by $h_j(x_i)$ where $j = 1, \dots, 2d + 1$ and the h_j are **strictly monotonic functions**.¹

The activation of the j th unit in the second hidden layer is given by

$$z_j = \sum_{i=1}^d \lambda_i h_j(x_i)$$

where $0 < \lambda_i < 1$ are constants.

¹A function is called **strictly monotonically increasing** if for all x and y such that $x \leq y$ one has $f(x) < f(y)$, so f preserves the order.

Implications for neural networks

The output y of the network is then given by

$$y = \sum_{j=1}^{2d+1} g(z_j)$$

where the function g is real and continuous.

Note that the function g depends on the particular function $y(x)$ which is to be represent, while the functions h_j do not.

This expression can be extended to a network with more than one output unit simply by modifying the above equation to give

$$y_k = \sum_{j=1}^{2d+1} g_k(z_j)$$

Implications for neural networks

Note that the **theorem only guarantees the existence** of a suitable network.

No actual examples of functions h_j or g are known, and there is no known constructive technique for finding them.

Limitations

While Kolmogorov's theorem is remarkable, its relevance to practical neural computing is at best limited.

There are two reasons for this:

- The functions h_j are far from being smooth. Indeed, it has been shown that if the functions h_j are required to be smooth then the theorem breaks down. The presence of nonsmooth functions in a network would lead to problems of extreme sensitivity the input variables. Smoothness of the network mapping is an important property in connection with the generalization performance of a network.

Limitations

- The second reason is that the function g depends on the particular function $y(x)$ which we wish to represent. This is the converse of the situation which we generally encounter with neural networks. Usually, we consider fixed activation functions, and then adjust the number of hidden units, and the values of the weights and biases, to give a sufficiently close representation of the desired mapping.

Limitations

In Kolmogorov's theorem the number of hidden units is fixed, while the activation functions depend on the mapping.

In general, if we are trying to represent an arbitrary continuous function then we cannot hope to do this exactly with a finite number of fixed activation functions since the finite number of adjustable parameters represents a finite number of degrees of freedom, and a general continuous function has effectively infinitely many degrees of freedom.

References

● Author *Title*, Editorial, year.